

Taller Unidad 2
Aplicación De Un Patrón Arquitectónico En Un Sistema
Fasttrack

Programa: Ingeniería De Sistemas
Asignatura: Arquitectura De Software
Profesor: Jesús Ariel Gonzales

Integrantes
María Sofia Aljure Herrera
Brayan Smith Bedoya

Corporación Universitaria Del Huila Corhuila
Neiva – Huila
Septiembre 2025

Contenido

Introducción.....	4
Justificación	4
Objetivos	4
Objetivo General	4
Metodología Design Thinking	5
1. Empatizar.....	5
2. Definir.....	5
3. Idear	5
4. Prototipar	5
5. Evaluar	5
Marco teórico.....	6
1. Arquitectura de software y principios de diseño.	6
2. Patrón Modelo–Vista–Controlador (MVC)	6
3. Patrón Cliente–Servidor	7
4. Arquitectura de microservicios	7
Desarrollo del trabajo	7
Patrones arquitectónicos definidos en el trabajo:	7
Selección y justificación del patrón adecuado	8
¿Qué es la arquitectura hexagonal?.....	9
¿Por qué usar la arquitectura hexagonal en FastTrack?	9
By Module y Hexagonal:.....	9
By Module:.....	10
Hexagonal:	10
¿Por qué Hexagonal es mejor que By Module?	10
¿Cómo se hace el salto de By Module a Hexagonal?	10
By Module:.....	10
Hexagonal:	10
Aplicación del patrón al sistema	11
Priorización de funcionalidades – Método MoSCoW	11
Componentes y capas del sistema	12
Componentes principales arquitectura hexagonal	12
Tecnologías y lenguajes	14

Microfrontend	14
¿Qué es un microfrontend?	14
¿Cuál es su importancia?.....	14
Ventajas	15
Desventajas	15
Estructura del Microfrontend	15
Estructura general del proyecto.....	16
Estructura interna de un microfrontend (ejemplo: cliente-app)	16
Ejecución del microfrontend	17
Base de datos.....	17
Diagrama base de datos	18
Microservicio “Client-service”.....	19
Microservicio “Shipment-service”	19
Ambientes	21
Entorno QA (Quality Assurance).....	21
Entorno Deploy (Staging o Pre-Producción)	22
Entorno Producción (Production).....	22
Beneficios para los usuarios y el equipo de desarrollo	22
Diagrama de Casos de Uso	23
Diagrama de Clases	24
Diagrama de Paquetes	25
Diagrama de Secuencia	25
Conclusiones	26
Referencias	26

Tabla Ilustraciones

Ilustración 1 Arquitectura hexagonal.....	9
Ilustración 2 Arquitectura Microfrontend.....	15
Ilustración 3 Client-service.....	19
Ilustración 4 Shipment-service	19
Ilustración 5 Fleet-service	20
Ilustración 6 Pay-Service.....	20
Ilustración 7 Notification-Service	21

Ilustración 8 People-service	21
Ilustración 9 Diagrama de Casos de Uso.....	23
Ilustración 10 Diagrama de Clases	24
Ilustración 11 Diagrama de Paquetes	25
Ilustración 12 Diagrama de Secuencia	25
Tabla 1 Método MoSCoW	11
Tabla 2 Entorno comando	18

Introducción

FastTrack es una empresa de logística que quiere tener una plataforma digital para gestionar envíos, clientes, flota y personal. Antes de avanzar con cualquier desarrollo, necesita tener claro cómo va a estar organizado ese sistema, es decir, qué patrón arquitectónico va a usar para que todo funcione bien, crezca sin problemas y sea fácil de mantener.

En este trabajo se plantea cómo debería estructurarse la plataforma y se justifica la elección del patrón arquitectónico más adecuado, considerando tanto las necesidades actuales de la empresa como su proyección a futuro.

Justificación

FastTrack requiere una plataforma digital que le permita gestionar de manera ordenada sus envíos, clientes, flota y personal. La necesidad de definir un patrón arquitectónico antes de iniciar el desarrollo surge como una motivación central, ya que garantiza que el sistema pueda crecer sin descontrol, sea fácil de mantener y responda adecuadamente a las demandas de la empresa.

Este trabajo tiene relevancia práctica porque ofrece una propuesta de organización sólida para un sistema realista, evitando problemas de escalabilidad o rendimiento a futuro. Al mismo tiempo, posee valor académico, ya que fomenta la reflexión sobre la importancia de la arquitectura de software como base para la construcción de sistemas eficientes y sostenibles.

Objetivos

Objetivo General

Diseñar la estructura inicial de un sistema digital para la empresa de logística FastTrack, seleccionando el patrón arquitectónico más adecuado para organizar sus funciones de manera clara, escalable y fácil de mantener en el futuro.

Metodología de trabajo: Design Thinking

Metodología Design Thinking

Para el desarrollo del sistema FastTrack se aplican los principios del **Design Thinking**, una metodología centrada en comprender las necesidades de los usuarios y diseñar soluciones efectivas a partir de ellas. Este enfoque permite que el proyecto no solo cumpla con los objetivos técnicos, sino que también responda de forma real a las expectativas de clientes, administradores y personal operativo de la empresa.

El proceso se desarrolla a través de cinco etapas principales:

1. Empatizar

En esta fase se busca conocer a fondo a los usuarios del sistema, entendiendo sus rutinas, dificultades y expectativas.

En el caso de FastTrack, se analizan los roles de cliente, conductor y administrador para identificar problemas como la falta de información en tiempo real sobre los envíos, la dificultad de gestión de flota o los retrasos en notificaciones.

2. Definir

Con la información obtenida, se formulan los problemas principales que el sistema debe resolver.

Por ejemplo, la necesidad de una **plataforma unificada** que permita controlar envíos, notificar a los clientes, y administrar la flota de manera organizada y eficiente.

3. Idear

En esta etapa se generan ideas y posibles soluciones que respondan a los problemas definidos.

Para FastTrack, se proponen funcionalidades como **seguimiento en tiempo real**, **notificaciones automáticas**, **módulo de facturación** y **gestión independiente por microservicios**. Estas ideas se priorizan usando el método **MoSCoW**, asegurando que lo más importante se implemente primero.

4. Prototipar

Aquí se desarrollan versiones iniciales del sistema para visualizar cómo funcionará en la práctica.

En el proyecto, esto incluye **el diseño de los microfrontends** (cliente-app, envío-app, flota-app, etc.) y la creación de **diagramas de arquitectura** como los de clases, casos de uso y secuencia, que representan el flujo real del sistema.

5. Evaluar

Finalmente, se revisan los prototipos y se validan las ideas con usuarios o miembros del equipo.

En esta fase se identifican mejoras, se ajustan las funcionalidades y se preparan los entornos de prueba (QA y Deploy) para garantizar la calidad antes del lanzamiento oficial.

Aplicar el Design Thinking en FastTrack permitió al equipo comprender mejor las necesidades de los usuarios y definir una arquitectura flexible, modular y enfocada en ofrecer soluciones prácticas y escalables. Este enfoque asegura que la tecnología responda a las personas, y no al revés.

Marco teórico

1. Arquitectura de software y principios de diseño.

La arquitectura de software se entiende como la organización fundamental de un sistema, la definición de sus componentes, las relaciones entre ellos y los principios que guían su evolución. Bass, Clements y Kazman (2021) la describen como el esqueleto que soporta las cualidades de un sistema y orienta su crecimiento.

En el diseño arquitectónico se recomienda usar principios como la separación de responsabilidades, el encapsulamiento y la inversión de dependencias, con el fin de lograr sistemas más modulares, flexibles y fáciles de mantener (Microsoft, 2023a). Estos

principios contribuyen a que los cambios puedan realizarse en un módulo sin afectar a todo el sistema, favoreciendo la mantenibilidad y la escalabilidad.

2. Patrón Modelo–Vista–Controlador (MVC)

El patrón Modelo–Vista–Controlador (MVC) divide una aplicación en tres componentes principales:

- **Modelo**, que gestiona los datos y la lógica del negocio.
- **Vista**, que representa la interfaz con el usuario.
- **Controlador**, que actúa como intermediario entre modelo y vista, procesando entradas y actualizando salidas.

Según Gamma, Helm, Johnson y Vlissides (1995), MVC facilita el desacoplamiento entre la lógica de negocio y la interfaz de usuario, promoviendo la reutilización de componentes y la evolución independiente de cada parte. Este patrón ha sido ampliamente adoptado en aplicaciones web y de escritorio.

De acuerdo con Romero (2016), MVC es un referente dentro de los patrones arquitectónicos porque logra un reparto claro de responsabilidades y ha influido en el desarrollo de frameworks modernos en distintos lenguajes de programación.

3. Patrón Cliente–Servidor

El patrón Cliente–Servidor organiza el sistema en dos roles principales: los clientes, que realizan solicitudes, y el servidor, que responde a dichas peticiones y administra los recursos compartidos. Este modelo se caracteriza por centralizar los servicios, lo que facilita su control y seguridad (Tanenbaum & Van Steen, 2017).

Romero (2016) explica que este patrón constituye la base de la mayoría de los sistemas distribuidos actuales, ya que permite la interacción entre múltiples usuarios y un núcleo centralizado de datos, siendo especialmente útil en entornos donde se requiere disponibilidad compartida y comunicación a través de una red.

4. Arquitectura de microservicios

El estilo microservicios propone dividir una aplicación en servicios pequeños, autónomos e independientes, cada uno orientado a una capacidad de negocio dentro de un contexto delimitado (*bounded context*). Estos servicios se desarrollan, despliegan y escalan de manera independiente, comunicándose entre sí a través de interfaces bien definidas (Microsoft, 2023b).

En la práctica, cada microservicio corre en su propio proceso y puede gestionar su propio esquema de datos, lo que reduce dependencias y aumenta la independencia de desarrollo.

Esta arquitectura fomenta un acoplamiento débil entre servicios y una cohesión alta dentro de cada uno, favoreciendo la evolución continua de los sistemas (Microsoft, 2023c).

Desarrollo del trabajo

Patrones arquitectónicos definidos en el trabajo:

- **Modelo–Vista–Controlador (MVC):**

Es un patrón que separa la aplicación en tres capas: el **Modelo** (datos y lógica de negocio), la **Vista** (interfaz de usuario) y el **Controlador** (intermediario entre ambos). Es muy usado en aplicaciones web y de escritorio porque facilita la reutilización de componentes y la independencia de la interfaz.

Pero fue pensado para aplicaciones monolíticas; dificulta la escalabilidad horizontal; cambios en una capa suelen impactar en todo el sistema. aunque suele estar limitado a sistemas monolíticos poco escalables.

- **Cliente–Servidor:**

Divide el sistema en dos roles principales: el **cliente**, que realiza solicitudes, y el **servidor**, que procesa y responde a esas solicitudes. Es la base de la mayoría de aplicaciones distribuidas tradicionales y permite centralizar servicios y datos.

Sin embargo, cuando la carga recae sobre el servidor, tiene limitaciones de escalabilidad cuando los dominios de negocio crecen y no favorece el desarrollo ágil de nuevas capacidades.

- **Microservicios:**

Propone dividir la aplicación en servicios pequeños, autónomos e independientes, cada uno responsable de una capacidad de negocio. Estos servicios se despliegan y escalan de forma independiente, comunicándose a través de APIs o mensajería. Aunque una limitación es que implica mayor complejidad de infraestructura pero ofrece alta escalabilidad, mantenibilidad y flexibilidad para el crecimiento futuro.

Selección y justificación del patrón adecuado

FastTrack necesita gestionar envíos nacionales e internacionales, seguimiento en tiempo real, notificaciones a clientes y administración de flota y personal. Esto implica un sistema con múltiples dominios de negocio, alta demanda de disponibilidad y posibilidad de crecer con nuevas funcionalidades.

- En **escalabilidad**, los microservicios permiten que servicios críticos se escalen sin necesidad de aumentar toda la plataforma.
- En **mantenibilidad**, cada servicio puede evolucionar de manera independiente, reduciendo riesgos de fallos globales.
- En **distribución de responsabilidades**, cada equipo puede encargarse de un microservicio específico (Envíos, Clientes, Facturación).
- En **crecimiento futuro**, la empresa podrá añadir nuevos servicios sin alterar el núcleo ya existente.

Por lo tanto, la arquitectura de **Microservicios** es la más adecuada para el escenario de FastTrack.

Sin embargo, la elección de microservicios como arquitectura global plantea también la necesidad de definir cómo se organizará internamente cada servicio. Para este propósito, se ha decidido implementar la Arquitectura Hexagonal (puertos y adaptadores) como patrón de diseño dentro de cada microservicio. Esta estructura permite mantener el dominio desacoplado esto quiere decir que no depende directamente de cosas externas como: (frameworks, bases de datos, controladores REST), facilitando la mantenibilidad y la sustitución de tecnologías sin afectar la lógica central.

De este modo, la solución combina lo mejor de ambos enfoques:

- **Microservicios** para la organización global del sistema, asegurando escalabilidad y modularidad a nivel de negocio.
- **Hexagonal** para la organización interna de cada microservicio, asegurando un diseño limpio, flexible y preparado para cambios futuros.

¿Qué es la arquitectura hexagonal?

La arquitectura hexagonal, también conocida como patrón de puertos y adaptadores, es un estilo de diseño que coloca el dominio del negocio en el centro del sistema, rodeado por interfaces (puertos) que definen cómo se comunica con el exterior. Los elementos externos como bases de datos, APIs REST, sistemas de mensajería o frameworks se conectan a través de adaptadores, lo que permite que la lógica central no dependa directamente de la tecnología utilizada.

En otras palabras, el hexágono representa la idea de que el sistema puede conectarse con el mundo exterior de diferentes maneras sin que eso afecte su corazón.

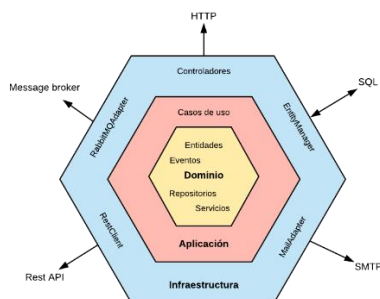


Ilustración 1 Arquitectura hexagonal

¿Por qué usar la arquitectura hexagonal en FastTrack?

1. **Mantiene el dominio independiente:** las reglas de negocio de no quedarán atadas a una base de datos o a un framework específico.
2. **Facilita el cambio de tecnologías:** si en el futuro se reemplaza una base de datos, una API o un protocolo de comunicación, solo se modifica el adaptador, sin tocar la lógica central.
3. **Mejora la mantenibilidad:** al estar desacoplado, el código es más limpio y fácil de probar.
4. **Encaja con los microservicios:** cada microservicio tendrá su propio hexágono, lo que asegura independencia real entre los distintos módulos de la plataforma.

By Module y Hexagonal:

En proyectos anteriores se utilizó la arquitectura **By Module (N-Capas)**, adecuada para sistemas pequeños pero limitada cuando el software debe escalar. Para el caso de FastTrack, se decidió dar el salto a la **arquitectura hexagonal por estos motivos:**

By Module:

- La aplicación se divide en paquetes o capas (controller, service, repository, entity).
- Cada capa depende de la siguiente: el controlador llama al servicio, el servicio al repositorio, y así sucesivamente.
- Es fácil de entender y rápido de implementar, pero crea un acoplamiento fuerte: el dominio termina dependiendo de detalles técnicos (como la base de datos o el framework).

Hexagonal:

- El núcleo del sistema es el dominio, que no depende de nada externo.
- Se definen puertos (interfaces) que el dominio necesita.
- Los adaptadores conectan esos puertos con el exterior (BD, APIs REST, mensajería, etc.).
- la lógica de negocio se mantiene independiente de frameworks y tecnologías.

¿Por qué Hexagonal es mejor que By Module?

1. **Independencia tecnológica:** en by module, si cambias la base de datos o el framework, normalmente afecta a todo el sistema. En hexagonal, solo cambias el adaptador.
2. **Mantenibilidad:** el dominio queda limpio y separado, lo que facilita pruebas unitarias sin necesidad de levantar toda la infraestructura.
3. **Escalabilidad:** al estar desacoplado, es más fácil transformar cada módulo en un microservicio independiente.
4. **Claridad en responsabilidades:** separa reglas de negocio (qué hace FastTrack) de infraestructura (cómo se conecta con el mundo).

¿Cómo se hace el salto de By Module a Hexagonal?

By Module:

```
src/  
├── controller/  
├── service/  
├── repository/  
└── entity/
```

En by module todo depende en cadena y el dominio queda acoplado a BD y frameworks.

Hexagonal:

```
src/  
├── domain/      # Núcleo del negocio (entidades, lógica, interfaces)  
│   ├── model/  
│   └── puerto/  
└──
```

└─ application/ # Casos de uso (orquestan el dominio)
└─ infrastructure/ # Adaptadores (REST, BD, config, excepciones)

En hexagonal el dominio define qué necesita, y la infraestructura se conecta como “enchufes” sin tocar el corazón del sistema.

El salto consiste en reubicar responsabilidades:

- Lo que antes estaba en repository pasa a ser un puerto de salida (interfaz en domain/puerto/out).
- Los servicios de negocio que estaban en service ahora son casos de uso en application/service.
- Los controladores (controller) pasan a ser adaptadores de entrada en infrastructure/rest/controller.
- Las entidades del dominio se separan en dos:
 - domain/model → entidades de negocio puras.
 - infrastructure/entity → entidades de persistencia.

Aplicación del patrón al sistema

Priorización de funcionalidades – Método MoSCoW

Para organizar el desarrollo de la plataforma FastTrack y garantizar que el equipo se enfoque en lo más importante desde el inicio, se utiliza el método **MoSCoW**, una técnica que permite clasificar las funcionalidades del sistema según su nivel de prioridad. Esto facilita una planificación más clara, evitando sobrecargas de trabajo y asegurando que el sistema cumpla primero con sus objetivos esenciales. Must Have (Debe tener) - Should Have (Debería tener) - Could Have (Podría tener)

Tabla 1 Método MoSCoW

FUNCIONALIDADES	MUST HAVE (DEBE TENER)	SHOULD HAVE (DEBERÍA TENER)	COULD HAVE (PODRÍA TENER)	WON'T HAVE (NO TENDRÁ POR AHORA)
Registro y gestión de clientes.	X			
Reportes sobre el estado de los envíos y desempeño de la flota.		X		
Creación y seguimiento de envíos.	X			

Administración de flota y conductores.	X			
Procesamiento de pagos y generación de facturas.	X			
Envío automático de notificaciones a clientes.		X		
Integración con servicios externos como Google Maps o WhatsApp.		X		
Aplicación móvil nativa (se prioriza primero la versión web).				X
Chat en vivo dentro de la plataforma.				X
Sistema de autenticación y roles de usuario (administrador, cliente, conductor).	X			

Componentes y capas del sistema

El sistema de FastTrack se implementará bajo la arquitectura de Microservicios, y cada microservicio adoptará el diseño Hexagonal (puertos y adaptadores).

Componentes principales arquitectura hexagonal

1. Domain (dominio):

- Contendrá las entidades de negocio y las interfaces (puertos) que definen cómo se comunica el núcleo con el exterior.
- 15 entidades:

Client / Cliente

Address / Dirección

Package / Paquete

Shipment / Envío

Tracking / Seguimiento

Notification / Notificación

Invoice / Factura

Payment / Pago

Vehicle / Vehículo

Route / Ruta

Driver / Conductor

Employee / Empleado

Role / Rol

User / Usuario

Support / Soporte

Application (casos de uso):

- Orquesta la lógica del dominio, representando los procesos clave de la empresa (ejemplo: registrar envío, generar factura, asignar vehículo, enviar notificación).
- Aquí se definen servicios de aplicación y comandos (DTOs de entrada).

Infrastructure (infraestructura):

- Proporciona los adaptadores que conectan el dominio con el mundo exterior:
 - **rest/controller** → APIs REST para exponer los servicios a clientes web y móviles.
 - **entity** → Entidades de persistencia (JPA/Hibernate).
 - **mapper** → Transformación entre entidades de dominio y DTOs de infraestructura.
 - **adaptador/out** → Implementaciones concretas de repositorios o integraciones externas (bases de datos, colas de mensajería, servicios externos).
 - **conf** → Configuración de Spring Boot y beans.

Resources:

- Configuración de la aplicación (application.yml), conexión a bases de datos, colas de mensajería y parámetros externos.

Tecnologías y lenguajes

Entorno de desarrollo:

- **Visual Studio Code (VSC)** → Editor principal para estructurar y desarrollar el proyecto.

Lenguaje y framework:

- **Java** como lenguaje de programación.
- **Spring Boot** como framework para implementar microservicios y aplicar el diseño hexagonal

Base de datos:

- **MySQL** (gestionado con **Docker** como servidor local).
- JPA/Hibernate como capa de persistencia dentro de los adaptadores.

Contenedores:

- **Docker** → Cada microservicio se empaquetará en un contenedor independiente para facilitar despliegue y escalabilidad.

Microfrontend

Los microfrontends, un término que apareció a por primera vez en el radar de ThoughtWorks a mediados del 2016 y que se introdujo como una tecnología en adopción durante el 2019; la cual pretende convertir la interfaz monolítica que usualmente se ofrece al cliente en algo similar a lo que se hace en el servidor con los Microservicios. Sin embargo, para lograr esto es importante entender algunos conceptos y establecer reglas o principios que sirvan como un estándar para facilitar el desarrollo de los mismos (Geers, 2019)

¿Qué es un microfrontend?

Casi siempre que alguien habla de microfrontends se dice que es adaptar los microservicios del lado del cliente y es cierto, sin embargo, una definición puede ser la que ofrece Cam Jackson en el sitio de Martin Fowler que dice: "An architectural style where independently deliverable frontend applications are composed into a greater whole" (Jackson, 2019) Pero más allá de esto, en los microfrontends lo que realmente se busca es entender cómo está estructurado un sitio web y cómo se relacionan sus partes, así que estos no son más que un estilo arquitectónico que pretende independizar en la medida de lo posible componentes, funcionalidades o características dentro de un sitio web.

¿Cuál es su importancia?

Para entender un poco más de la importancia de los microfrontends, es necesario contextualizar lo que sucede hoy en día. Ya se ha dicho que la web cambió y que ahora es moderna, pero esto no dice mucho, así que tomaremos el siguiente esquema donde nos

presentan tres arquitecturas que ahora son bien conocidas en el desarrollo de aplicaciones web.

Ventajas

Algunos de los beneficios que brindan puntualmente los microfrontends son:

1. Repositorios con código más cohesivo, fácil de mantener y en menores cantidades (spaghettiless).
2. Equipos empoderados, desacoplados y escalables.
3. Cambios en partes específicas de la interfaz de forma más rápida, sin afectar a las demás.

Desventajas

1. Algunas de las desventajas que hay en este estilo arquitectónico son:
2. Grandes retos para eliminar dependencias o comunicar los microfrontends.
3. Duplicación de dependencias.
4. Mayor cantidad de archivos que el cliente requiere descargar.
5. Añade complejidad al ciclo de integración y despliegue continuo
6. Requiere que el equipo maneje bien el tema.

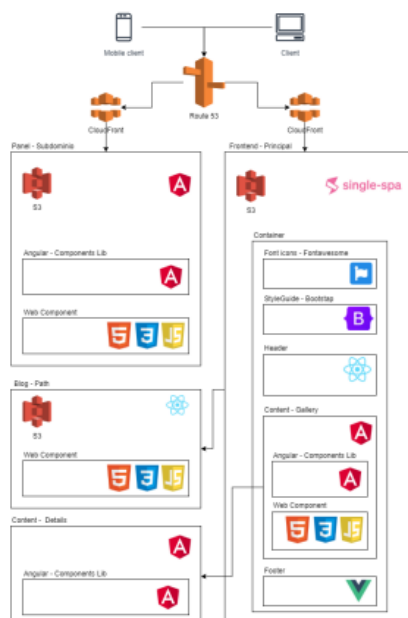


Ilustración 2 Arquitectura Microfrontend

Estructura del Microfrontend

En el proyecto **FastTrack-Frontend**, la arquitectura de microfrontends se implementa siguiendo la misma filosofía modular usada en el backend con microservicios. Cada dominio funcional de negocio (clientes, envíos, flota, notificaciones, pagos, etc.)

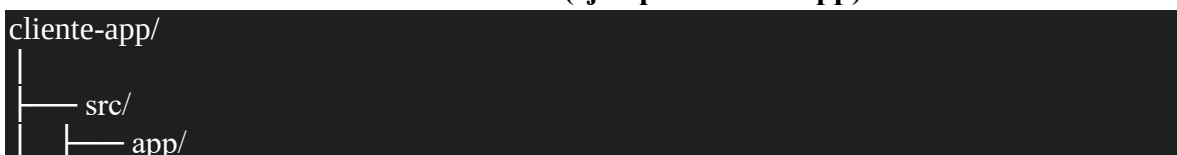
posee su propio frontend independiente, desarrollado con Angular, lo que permite mantener una separación total entre módulos y equipos de trabajo.

Estructura general del proyecto



Cada carpeta (por ejemplo, cliente-app, envio-app, flota-app) representa un **microfrontend** completamente independiente, con su propio entorno Angular, configuración y dependencias. Esto significa que cada aplicación puede desarrollarse, probarse y desplegarse por separado, manteniendo coherencia con los microservicios del backend.

Estructura interna de un microfrontend (ejemplo: cliente-app)



components/	# Componentes reutilizables de la vista
models/	# Modelos o interfaces TypeScript
pages/	# Páginas principales del módulo
services/	# Servicios para consumo de APIs REST
app-routing.module.ts	# Rutas del microfrontend
assets/	# Recursos estáticos (imágenes, íconos, estilos)
environments/	# Configuración de entornos (dev, prod)
index.html	# Punto de entrada HTML
main.ts	# Punto de arranque Angular
angular.json	# Configuración del proyecto Angular
package.json	# Dependencias y scripts específicos del microfrontend
README.md	

Ejecución del microfrontend

Cada microfrontend del proyecto FastTrack se ejecuta de forma independiente, manteniendo su propia configuración dentro de su carpeta correspondiente. En cada módulo (como cliente-app, envio-app, flota-app, pago-app o notificacion-app) se encuentran los archivos `angular.json`, `package.json` y la carpeta `src/`, donde están definidos los componentes visuales, los servicios que se comunican con los microservicios del backend y las configuraciones de entorno con las rutas de las APIs.

Para ejecutar un microfrontend, se deben instalar primero sus dependencias con el comando `npm install` y luego iniciarlo con `ng serve`, asignándole un puerto diferente a cada uno. Por ejemplo, el frontend de clientes puede ejecutarse en el puerto **4201**, el de envíos en **4202**, el de flota en **4203**, el de pagos en **4204** y el de notificaciones en **4205**. Esta distribución por puertos permite que todos los módulos funcionen al mismo tiempo sin interferir entre sí.

Cada microfrontend se comunica directamente con su microservicio correspondiente del backend, enviando y recibiendo datos mediante peticiones HTTP a través de los servicios definidos en Angular. De esta forma, cada módulo puede desarrollarse, probarse o actualizarse sin afectar a los demás.

El **gateway-app** cumple el papel de punto de entrada principal de la plataforma. Es el encargado de integrar y mostrar todos los microfrontends en una sola interfaz unificada. Cuando el gateway se ejecuta (por ejemplo, en el puerto **4200**), carga los demás microfrontends según las rutas o funciones que el usuario necesite, haciendo que el sistema completo funcione como una sola aplicación, aunque internamente esté formado por varios módulos independientes.

Base de datos

En este proyecto, la arquitectura está organizada en **microservicios independientes**, donde cada uno cumple una función específica dentro del sistema y posee su propia **base de datos**.

MySQL desplegada mediante contenedores Docker. Esta separación garantiza una mayor escalabilidad, mantenimiento individual y aislamiento de fallos entre los servicios.

Cada microservicio —como Client-service (gestión de clientes), Shipment-service (control de envíos), Fleet-service (administración de flota), Pay-service (procesamiento de pagos), Notification-service (envío de notificaciones) y People-service (gestión de personal)— corre en un puerto distinto, lo que permite su comunicación a través de APIs sin generar conflictos de red.

Para manejar los distintos **entornos de ejecución**, se han definido tres archivos de configuración de Docker Compose:

- docker-compose.qa.yml para el entorno **QA (Quality Assurance)**, usado en pruebas y validaciones previas al despliegue.
- docker-compose.deploy.yml para el entorno de **Staging o Deploy**, que simula el entorno de producción para pruebas finales.
- docker-compose.prod.yml para el entorno de **Producción**, donde la aplicación se ejecuta de forma estable y accesible para los usuarios finales.

Cada uno de estos archivos configura los contenedores de los microservicios y sus respectivas bases de datos MySQL, permitiendo levantar el entorno adecuado con un solo comando. La ejecución se realiza de la siguiente manera según el entorno:

Tabla 2 Entorno comando

Entorno	Comando
QA	docker-compose -f docker/docker-compose.qa.yml up -d
Producción	docker-compose -f docker/docker-compose.prod.yml up -d
Deploy (Staging)	docker-compose -f docker/docker-compose.deploy.yml up -d

De esta forma, la infraestructura basada en Docker proporciona una gestión clara, modular y automatizada de todos los servicios y sus dependencias, manteniendo la coherencia entre entornos y facilitando el ciclo completo de desarrollo, prueba y despliegue.

Diagrama base de datos

Microservicio “Client-service”

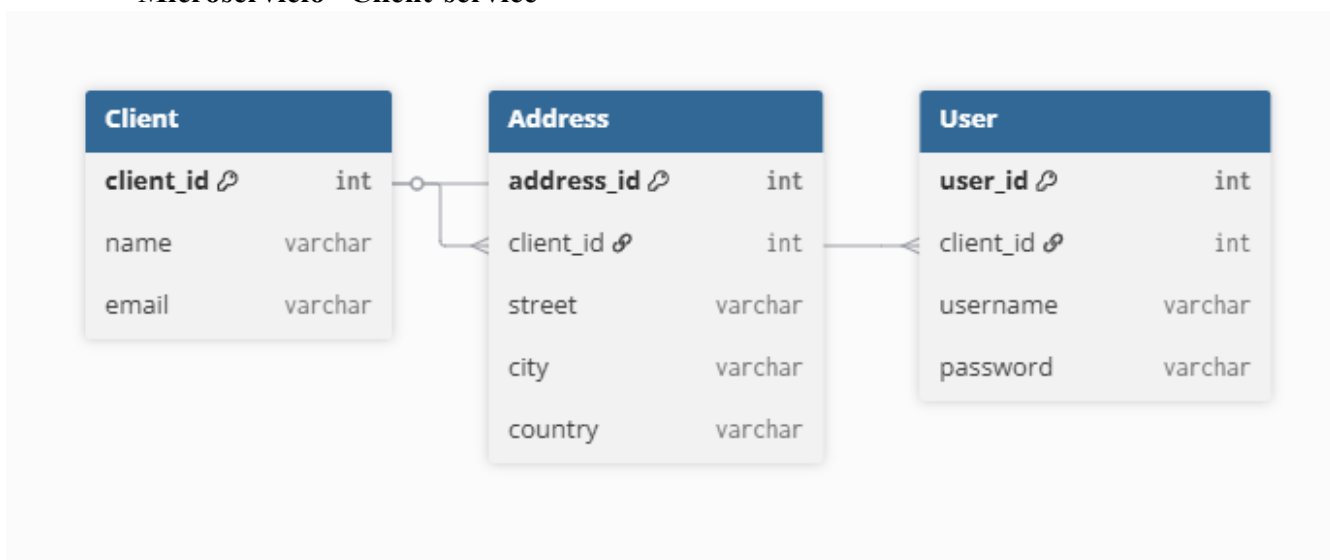


Ilustración 3 Client-service

Microservicio “Shipment-service”

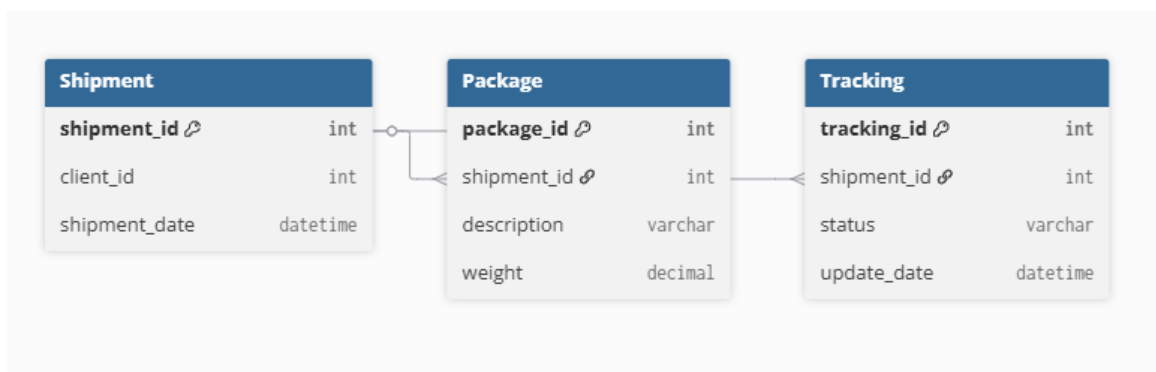


Ilustración 4 Shipment-service

Microservicio: “Fleet-service”

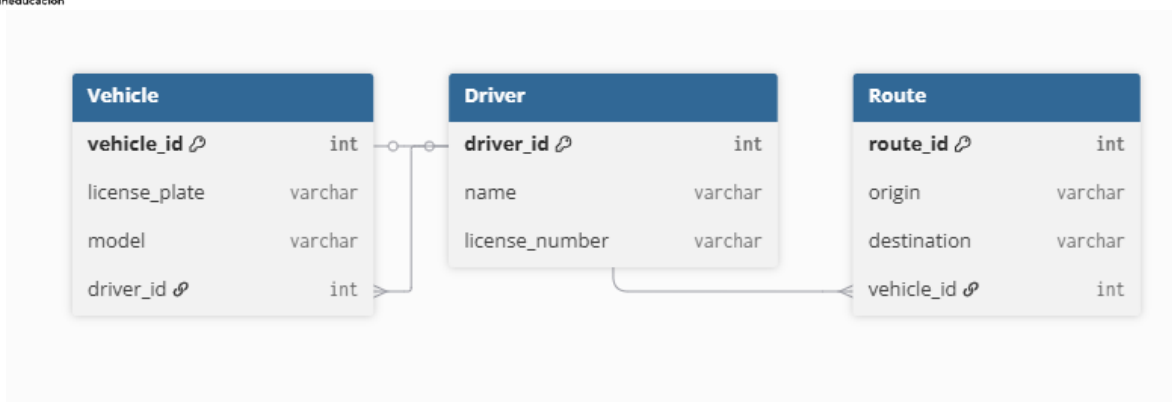


Ilustración 5 Fleet-service

Microservicio: “Pay-Service”

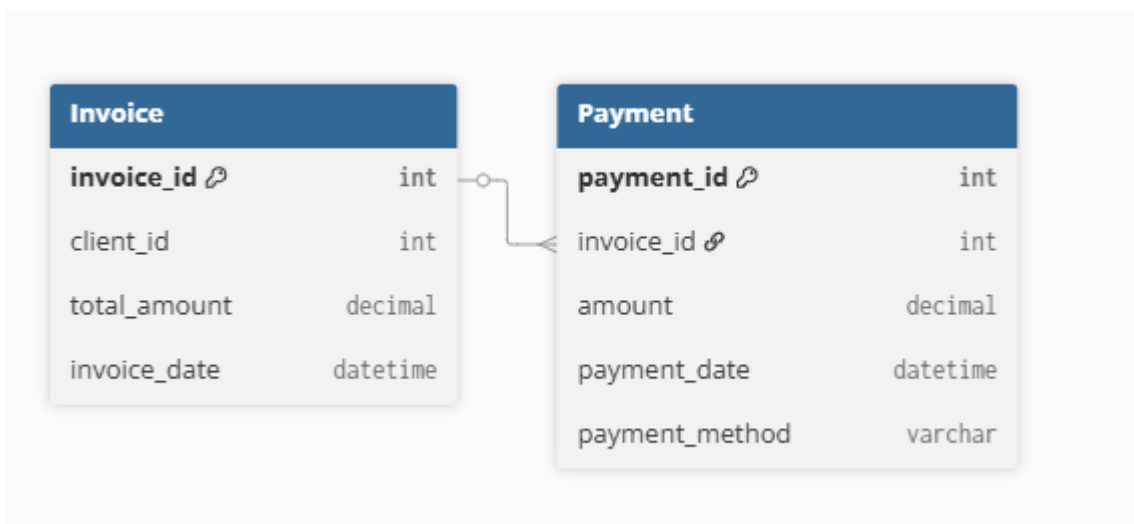


Ilustración 6 Pay-Service

Microservicio: “Notification-Service”

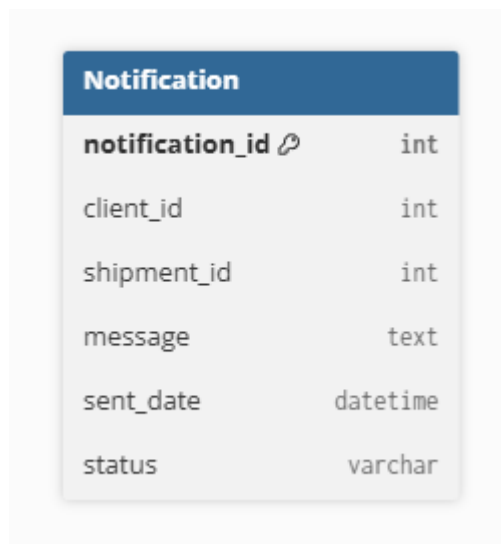


Ilustración 7 Notification-Service

Microservicio: “People-service”

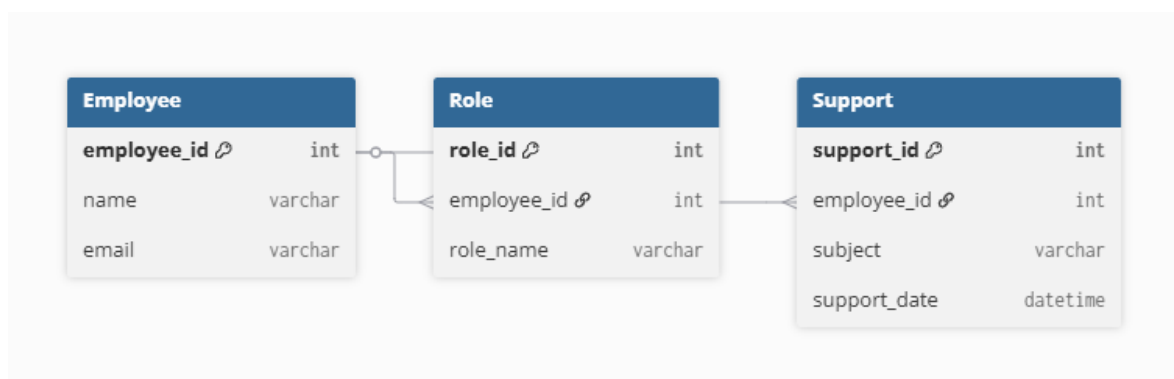


Ilustración 8 People-service

Ambientes

Entorno QA (Quality Assurance)

El entorno QA es donde se llevan a cabo las pruebas de calidad y verificación funcional de cada microservicio antes de pasar a etapas superiores. En este entorno, los desarrolladores y el equipo de pruebas evalúan nuevas características, corrigen errores y verifican que la comunicación entre los microservicios y sus respectivas bases de datos MySQL sea correcta. Aquí se utiliza una base de datos temporal con datos de prueba, permitiendo realizar pruebas sin comprometer información real. Además, el entorno QA suele mantener activado el modo *debug* para facilitar la detección de fallos y el seguimiento de logs en tiempo real, garantizando así la estabilidad del sistema antes del despliegue.

Entorno Deploy (Staging o Pre-Producción)

El entorno Deploy, también conocido como Staging, es una réplica casi exacta del entorno de producción y actúa como una fase de preparación previa al lanzamiento oficial. En este entorno se ejecutan las pruebas de aceptación del usuario (UAT) y las pruebas de rendimiento, asegurando que todo el ecosistema de microservicios —incluyendo las bases de datos MySQL, APIs y servicios externos— funcione correctamente bajo condiciones reales. Aquí se utiliza una configuración más optimizada y segura que en QA, pero sin exponer datos productivos. Es el punto de validación final donde el equipo técnico y los stakeholders confirman que la aplicación está lista para ser desplegada de forma estable en producción.

Entorno Producción (Production)

El entorno **Producción** es el espacio donde el sistema se ejecuta de manera estable y está disponible para los **usuarios finales**. En este entorno, los microservicios se despliegan con configuraciones optimizadas para el **rendimiento, la seguridad y la escalabilidad**, utilizando bases de datos MySQL productivas con copias de seguridad, monitoreo constante y medidas de protección avanzadas. No se permite el modo debug ni cambios directos en el entorno; cualquier actualización pasa previamente por QA y Deploy. Producción es el entorno crítico donde la fiabilidad y la disponibilidad del sistema son la máxima prioridad, garantizando una experiencia fluida, segura y consistente para los clientes y usuarios del sistema.

Beneficios para los usuarios y el equipo de desarrollo

Para los usuarios:

- Plataforma disponible en web y móvil.
- Respuesta rápida gracias a la escalabilidad de microservicios.
- Notificaciones en tiempo real para el seguimiento de paquetes.

Para el equipo de desarrollo:

- Cada microservicio puede ser desarrollado y mantenido por un equipo independiente.
 - El dominio está aislado de la infraestructura, lo que facilita pruebas y cambios tecnológicos.
 - Nueva funcionalidad se puede añadir creando un nuevo microservicio o adaptador, sin afectar al resto del sistema.
 - Escalabilidad horizontal: servicios críticos como Tracking o Notifications pueden escalarse sin necesidad de aumentar toda la plataforma.
- Representación gráfica con PlantUML**

Diagrama de Casos de Uso

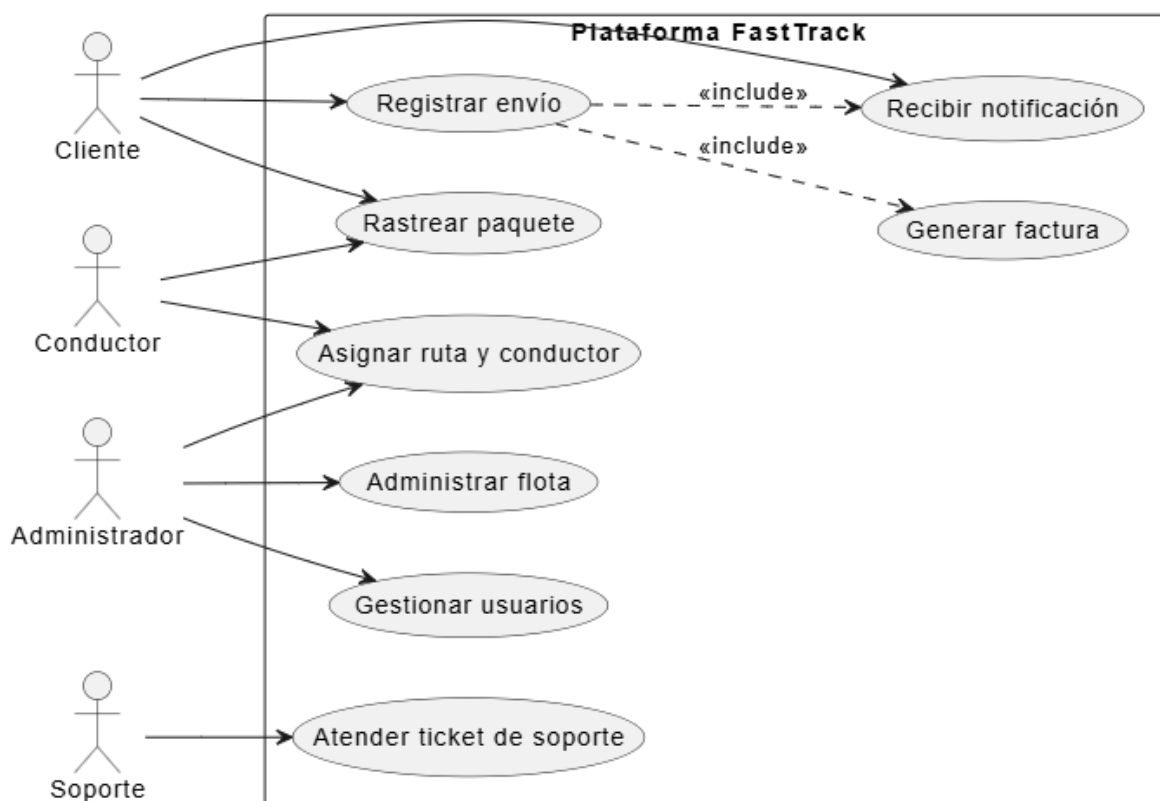


Ilustración 9 Diagrama de Casos de Uso

Este diagrama muestra las principales funciones que ofrece la plataforma FastTrack a los diferentes actores del sistema: clientes, administradores, conductores y soporte. Se representan acciones como registrar un envío, rastrear paquetes, recibir notificaciones, administrar la flota o atender tickets de soporte. Sirve para entender qué puede hacer cada usuario dentro de la plataforma y cómo se relaciona con las funcionalidades principales.

Diagrama de Clases

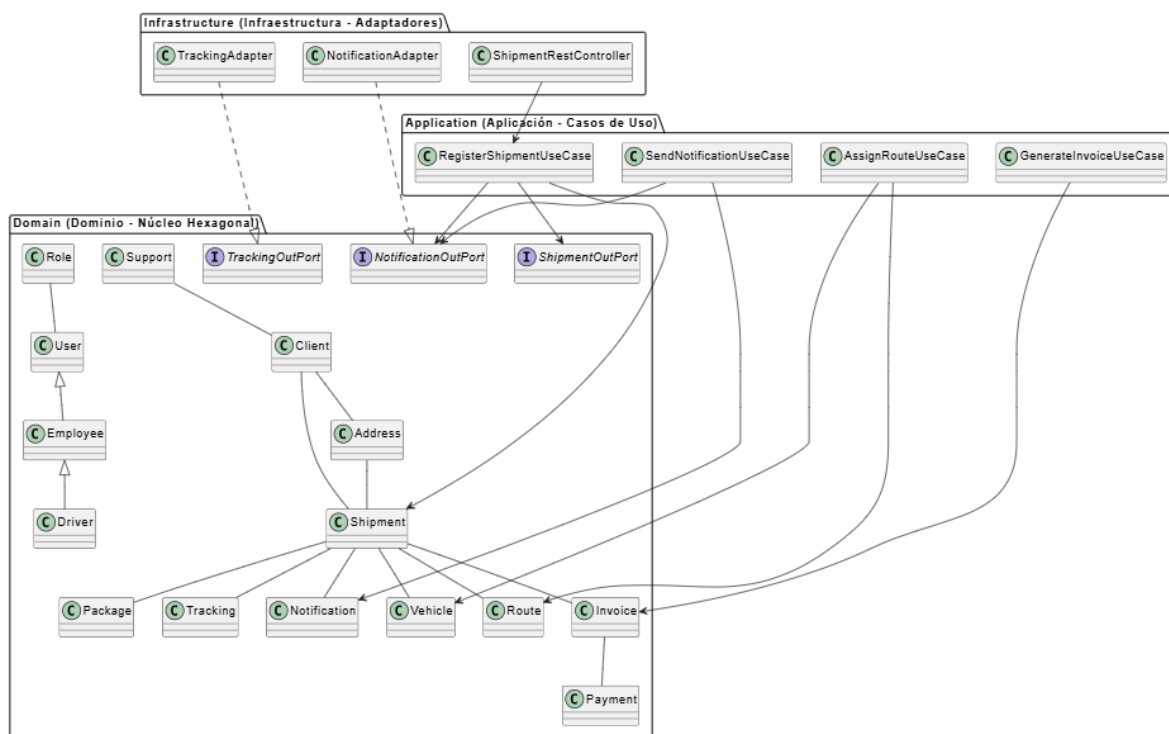


Ilustración 10 Diagrama de Clases

El diagrama de clases refleja cómo está organizado el sistema por dentro. Se divide en tres partes:

- **Dominio:** contiene las entidades principales como Cliente, Envío, Paquete, Factura, Vehículo o Usuario.
- **Aplicación:** maneja los procesos clave, por ejemplo, registrar un envío, asignar una ruta o generar una factura.
- **Infraestructura:** conecta el sistema con el exterior mediante adaptadores, como controladores web o notificaciones.

La idea es que las reglas de negocio (el núcleo) estén separadas de la tecnología, para que el sistema sea **más fácil de mantener y escalar**.

Diagrama de Paquetes

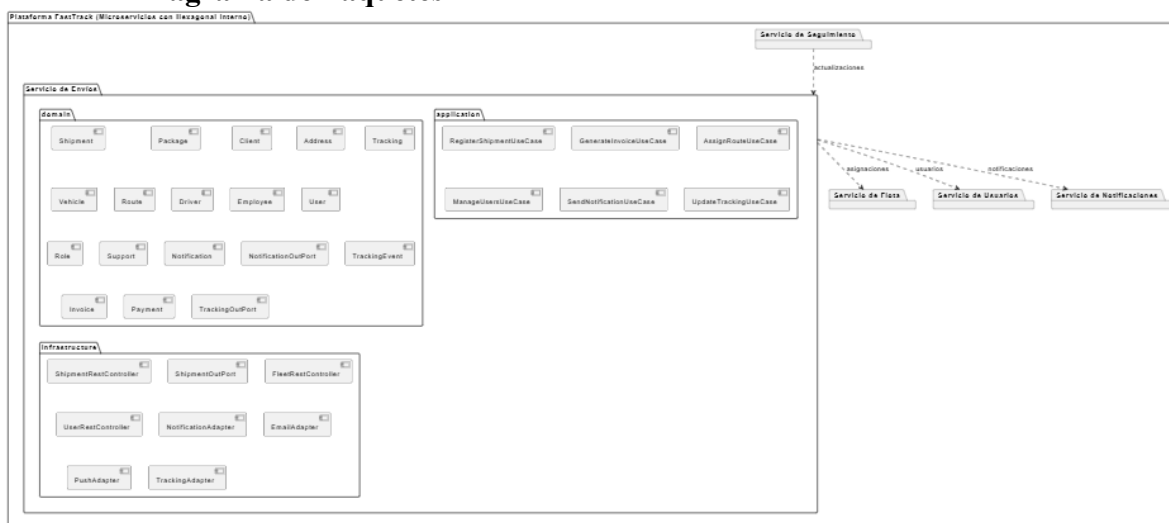


Ilustración 11 Diagrama de Paquetes

Aquí se representa la plataforma completa dividida en microservicios, cada uno encargado de un área específica: Envíos, Flota, Usuarios, Notificaciones y Seguimiento. Cada microservicio tiene su propia estructura interna (dominio, aplicación e infraestructura), lo que le da independencia para desarrollarse y crecer. Además, el diagrama muestra cómo los microservicios se comunican entre sí, por ejemplo, Envíos con Notificaciones o Flota. Este diseño garantiza organización, independencia y escalabilidad.

Diagrama de Secuencia

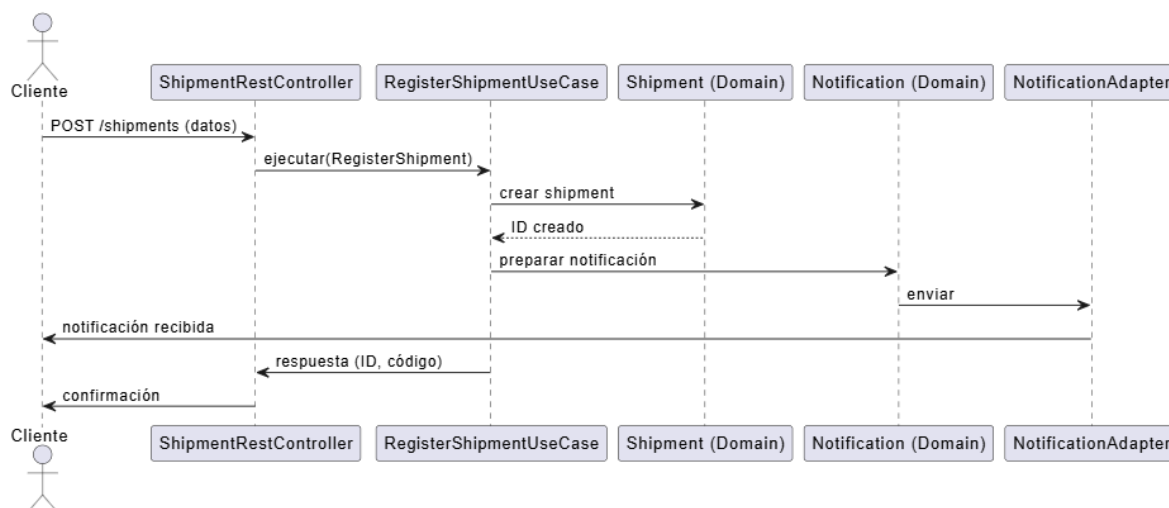


Ilustración 12 Diagrama de Secuencia

Este diagrama explica paso a paso cómo ocurre el proceso de registrar un envío. Primero, el cliente hace la solicitud en la plataforma; luego, el controlador recibe los datos y activa el caso de uso que crea el envío en el sistema. Después, se genera una notificación que es

enviada al cliente mediante un adaptador. Finalmente, el sistema responde confirmando el envío registrado. En resumen, muestra el flujo de comunicación entre los actores y las partes del sistema durante una operación concreta.

/shipments: es la ruta o “endpoint” de la API que maneja los envíos.

“POST /shipments” es la acción de un cliente que llena un formulario o manda datos para crear un nuevo envío en el sistema.

Conclusiones

El proyecto de FastTrack demuestra que una buena organización del sistema desde el inicio es clave para evitar problemas en el futuro. Al definir una arquitectura sólida, la empresa no solo garantiza que su plataforma funcione bien hoy, sino que también podrá crecer sin desorden, adaptarse a nuevas tecnologías y mantenerse estable con el paso del tiempo.

La combinación de microservicios con la arquitectura hexagonal le da al sistema una estructura muy flexible. Gracias a esto, cada parte del proyecto puede mejorar o cambiar sin afectar al resto, permitiendo que distintos equipos trabajen de forma independiente. En pocas palabras, el sistema está hecho para evolucionar y responder rápido a las necesidades de los usuarios.

Este trabajo refleja el esfuerzo de aplicar principios modernos de desarrollo para lograr un sistema que no solo sea funcional, sino también fácil de mantener y escalar. La propuesta de FastTrack muestra una visión clara: construir una plataforma tecnológica ordenada, confiable y pensada tanto para quienes la usan como para quienes la desarrollan.

Referencias

- Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4th ed.). Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Microsoft. (2023a). *Architectural principles*. Microsoft Learn.
<https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/architectural-principles>
- Microsoft. (2023b). *Microservice application design*. Microsoft Learn.
<https://learn.microsoft.com/es-es/dotnet/architecture/microservices/multi-container-microservice-net-applications/microservice-application-design>
- Microsoft. (2023c). *Microservices architecture style*. Microsoft Learn.
<https://learn.microsoft.com/es-es/azure/architecture/guide/architecture-styles/microservices>

- Romero, J. R. (2016). *Arquitectura de software: patrones arquitectónicos*. Universidad de Alicante, Repositorio Institucional (RUA).
<https://rua.ua.es/entities/publication/4b531331-9d6d-43a8-9ed7-5d4b3cd4beb8>
- Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed systems: Principles and paradigms* (3rd ed.). Pearson.
- Geers, M. (2019, Agosto 12). Micro Frontends - *Extendiendo la idea de microservicio al desarrollo frontend*. *Micro Frontends*. <https://micro-frontends-es.org/>.
- Jackson, C. (2019, Junio 19). *Micro Frontends*. Martinowler.Com.
<https://martinfowler.com/articles/micro-frontends.html>
- Arango Amaya, L.D. (2021). ARQUITECTURAS DE MICRO FRONTENDS, COMPONENTES WEB Y LIBRERÍAS DE COMPONENTES [Trabajo de grado profesional]. Universidad de Antioquia, Medellín, Colombia.